

Ciberseguridad Sostenible en el Borde: Reducción de la Huella de Carbono en la Inspección de APIs mediante Cómputo Verde y Teoría de Juegos

Equipo *Stochastic Green Edge* (SGE)

Computación Sostenible y Ciberseguridad Cloud

August 8, 2026

Resumen

El sector de las Tecnologías de la Información y la Comunicación (TIC) y, en particular, los centros de datos, representan una fracción creciente del consumo eléctrico mundial y de las emisiones de gases de efecto invernadero [1, 2]. Dentro de esta carga, la práctica de seguridad dominante —someter el 100% del tráfico entrante de una API a inspección profunda— impone un consumo energético elevado, constante e *invariante al riesgo*, un sobreprocesamiento que contradice los principios del cómputo verde. Este trabajo aborda la ciberseguridad desde la óptica de la **sostenibilidad**: presentamos **Stochastic Green Edge** (SGE), una defensa perimetral de dos capas —un filtro *Edge* de baja potencia y una función *Serverless* de inspección profunda— que activa el cómputo costoso *solo cuando es energéticamente justificable*. La decisión se optimiza como un equilibrio de Nash que iguala el *Joule marginal invertido con la seguridad marginal obtenida*. En una simulación de 2,000 peticiones, SGE reduce el consumo energético en un 85.9% (de 30,010 a 4,225 Joules) y, en consecuencia, la **huella de carbono en la misma proporción**: a escala de una API de 10^9 peticiones mensuales, ello equivale a evitar del orden de 2 **toneladas de CO₂ al año**, manteniendo una mitigación de amenazas del 94.38%. Mostramos que reservar el cómputo intensivo para la fracción de tráfico realmente sospechosa es un principio de ecodiseño eficaz para alinear la seguridad con la descarbonización.

Palabras clave: cómputo verde, huella de carbono, eficiencia energética, computación sostenible, edge computing, serverless, equilibrio de Nash, seguridad de APIs, inyección de prompts, AWS, ESG, B2B SaaS.

1 Introducción

La sostenibilidad del cómputo se ha convertido en una preocupación de primer orden: los centros de datos consumen ya un porcentaje significativo de la electricidad mundial y su demanda crece con la digitalización [1]. El *cómputo verde* (*green computing*) persigue reducir esa energía y las emisiones asociadas mediante el ecodiseño del

software: eliminar el sobreprocesamiento, aprovechar recursos de baja potencia y desplazar la carga hacia donde la energía es más limpia [2, 3].

La ciberseguridad de APIs es, paradójicamente, un foco de derroche energético oculto. La práctica habitual somete *cada* petición a una inspección profunda —sanitización exhaustiva, validación criptográfica, análisis de payload— con independencia de su riesgo real. Este gasto uniforme maximiza la cobertura, pero desperdicia energía (y emite CO₂) procesando de forma intensiva un tráfico que, en su gran mayoría, es legítimo o trivialmente malicioso. El problema es que la fracción verdaderamente peligrosa —ataques dirigidos que imitan tráfico legítimo, como la *inyección de prompts* (*prompt injection*) contra APIs respaldadas por modelos de lenguaje— **no es directamente observable**.

El costo ambiental de la seguridad “siempre al máximo”. Las defensas tradicionales —cortafuegos de aplicaciones web (WAF), pasarelas de API y tuberías de inspección profunda— se aprovisionan para procesar *el 100% del tráfico entrante a su máxima capacidad*: cada petición atraviesa el conjunto completo de reglas de firma, validación criptográfica y, cada vez más, análisis de contenido basado en aprendizaje automático o en modelos de lenguaje. Esta postura de “inspección total y constante” mantiene la CPU y los aceleradores en alta utilización de forma permanente, con independencia de que en torno al 95% del tráfico sea benigno o trivialmente malicioso. El resultado es un consumo eléctrico sostenido y, por la ecuación (2), una línea base de carbono elevada y constante —un desperdicio energético masivo dedicado a examinar exhaustivamente peticiones que no lo requieren. La irrupción de las APIs respaldadas por modelos de lenguaje agrava el problema: detectar una inyección de prompts puede exigir invocar un modelo, elevando el costo energético *por inspección* en varios órdenes de magnitud. SGE invierte este paradigma: un filtro *Edge* de baja potencia resuelve de forma económica la mayoría benigna y, guiado por el equilibrio de Nash, *solo* escala a la inspección profunda costosa la fracción sospechosa, recortando la energía —y la huella de carbono— de la defensa en un 85.9% sin degradar de forma apre-

ciable la protección.

Este artículo reformula la decisión de “inspeccionar o no” como un **problema de sostenibilidad**: minimizar la energía (y la huella de carbono) sujeta a un nivel de seguridad aceptable. Nuestra tesis es que la inspección profunda debe tratarse como un recurso energético escaso y asignarse *selectivamente*. Para hacerlo de forma óptima y robusta ante un adversario estratégico, modelamos la decisión como un juego no cooperativo y la resolvemos con el equilibrio de Nash en estrategias mixtas [5, 6], que actúa como el *mecanismo* para lograr el fin sostenible. Nuestras contribuciones son:

- Un **modelo de sostenibilidad** de la defensa perimetral que integra un modelo energético por capa y un modelo de contabilidad de carbono.
- Una **política de asignación energética** óptima que reserva el cómputo intensivo para el tráfico de alto riesgo, derivada del equilibrio de Nash.
- Una **cuantificación de la huella de carbono** evitada, con figuras y estimaciones reproducibles a escala real.

2 Objetivos y Alcance del Proyecto

2.1 Objetivo general

Diseñar, implementar y validar un **motor de decisión estocástico basado en teoría de juegos** que realice la *intercepción eficiente y segura de amenazas* sobre APIs e infraestructuras *Edge*, maximizando la tasa de detección efectiva mientras **minimiza el consumo energético y la huella de carbono** asociados a la inspección de tráfico, como alternativa cuantificablemente más sostenible que la inspección profunda continua.

2.2 Objetivos específicos

1. **(OE-1) Lógica estocástica y de teoría de juegos.** Formalizar el juego Defensor–Atacante, derivar el *equilibrio de Nash* en estrategias mixtas y materializar una política de inspección intermitente y adaptativa, reproducible bajo semilla fija (*seed-7*).
2. **(OE-2) Capa de seguridad en APIs.** Construir el módulo de filtrado/inspección sobre HTTP/REST, con inspección de *payloads* y *prompts* para detectar *prompt injection* y abuso de peticiones.
3. **(OE-3) Medición y reducción del consumo energético.** Instrumentar la telemetría de recursos y su conversión a huella de carbono (gCO₂eq), integrando el coste energético en el *payoff* del Defensor.

4. **(OE-4) Validación y benchmarking.** Comparar SGE frente a métodos convencionales (inspección continua y muestreo uniforme fijo), reportando eficiencia energética *vs.* tasa de detección (frontera de Pareto) bajo *seed-7*.

2.3 Alcance (*scope*)

Esta fase persigue un **prototipo funcional validado experimentalmente**. **Dentro del alcance:** el algoritmo estocástico (Nash) reproducible (*seed-7*); el módulo de filtrado de peticiones/APIs sobre HTTP/REST; la inspección de *payloads/prompts*; las métricas de consumo energético y su traducción a gCO₂eq; y el prototipo funcional con *benchmarking* reproducible. **Fuera del alcance** en esta fase quedan el despliegue en clústeres multirregión a gran escala, las interfaces gráficas complejas de administración, el soporte para protocolos industriales propietarios no HTTP/REST (gRPC, MQTT, OPC-UA, Modbus) y la certificación/*hardening* productivo. **Entregables clave:** artefactos de código (motor de decisión estocástico y proxy/gateway de inspección), documentación técnica y modelo matemático de la estrategia de inspección, y reporte de *benchmarks* (eficiencia energética *vs.* tasa de detección de amenazas).

3 Estado del Arte

La seguridad de aplicaciones en la nube se ha estructurado en torno a varias familias de controles. En el perímetro operan los *cortafuegos de aplicaciones web* (WAF) basados en firmas y reglas gestionadas, complementados con *limitación de tasa* (*rate-limiting*) y listas de reputación de IP. Frente a la denegación de servicio se despliegan servicios de mitigación volumétrica y redes de distribución de contenido. Un segundo estrato lo forman los detectores de anomalías basados en aprendizaje automático y las plataformas de gestión de postura (*CSPM/CNAPP*), que buscan configuraciones erróneas y comportamientos anómalos. Más recientemente, el auge de las aplicaciones respaldadas por modelos de lenguaje ha motivado *barreras de contenido* (*guardrails*) específicas contra la inyección de prompts [7, 8]. Paralelamente, la comunidad académica ha aplicado la teoría de juegos a la seguridad de redes, modelando la interacción atacante–defensor [5, 6].

Pese a su madurez, el estado del arte comparte tres limitaciones estructurales. Primera, es predominantemente **reactivo**: los controles actúan tras observar una firma o una anomalía, y su diseño optimiza la *cobertura* de detección, no el *costo* de defenderse. Segunda, trata la profundidad de inspección como un recurso **gratuito e ilimitado**: la recomendación por defecto es inspeccionar todo el tráfico a máxima profundidad, sin un modelo explícito del gasto computacional o energético

que ello implica. Tercera, y como corolario, la **eficiencia energética y la huella de carbono de la propia defensa están ausentes** del proceso de diseño; los marcos de referencia industriales priorizan riesgo y cumplimiento, pero no la sostenibilidad del mecanismo de mitigación [9, 10]. Los trabajos de seguridad basados en teoría de juegos, por su parte, rara vez incorporan el costo energético como término de la función de utilidad. SGE se sitúa precisamente en esa brecha.

3.1 Panorama de amenazas y mitigación en la nube (AWS)

La Tabla 1 sintetiza diez de las clases de ataque más frecuentes contra cargas en la nube y su mitigación con servicios nativos de Amazon Web Services (AWS), según los marcos de referencia vigentes [9, 8, 7, 10]. La mayoría de estos controles operan bajo la premisa de inspección exhaustiva; SGE es complementario y aporta la capa de *eficiencia* de la que carecen.

3.2 El estado del arte aplicado a SGE

SGE rompe el paradigma reactivo y “siempre al máximo” descrito arriba en tres sentidos. **(i) De reactivo a proactivo-económico:** en lugar de decidir tras detectar, SGE decide *a priori* qué peticiones merecen el gasto de detección, tratando la inspección profunda como un recurso escaso. **(ii) Filtrado estocástico de inyección:** para el tráfico sospechoso —en particular los payloads compatibles con inyección de prompts o de código— SGE no aplica una regla fija y explotable, sino una *política aleatorizada* derivada del equilibrio de Nash, que un adversario racional no puede eludir de forma sistemática. Así, la mayoría del tráfico malicioso ruidoso se neutraliza en el borde y solo la fracción sigilosa escala al motor costoso. **(iii) Descarga del servidor principal:** al resolver el 94.65% de las peticiones en la capa Edge de baja potencia, SGE reduce drásticamente la carga sobre las funciones de inspección profunda del núcleo, con el consiguiente ahorro de cómputo y de emisiones. En síntesis, SGE no sustituye a los controles de la Tabla 1, sino que los dota de una capa de *eficiencia energética* ausente en el estado del arte.

4 Marco de Cómputo Verde

4.1 Modelo energético por capa

SGE distingue dos capas de infraestructura con costos energéticos radicalmente asimétricos, medidos en unidades relativas de Joules/ciclos [3]:

$$C_{\text{edge}} = 1 \ll C_{\text{exec}} = 15 < C_{\text{cold}} = 25. \quad (1)$$

El filtro *Edge* es una función de baja potencia y paso rápido; la función *Serverless* realiza inspección profunda a un costo entre $15\times$ y $25\times$ mayor, penalización esta última debida al *arranque en frío* (*cold start*) tras un periodo de inactividad [4]. Esta relación 1:15:25 es el núcleo del argumento verde: *cada petición que se resuelve en el Edge en lugar de escalar ahorra entre 14 y 24 unidades de energía*.

4.2 Modelo de huella de carbono

La huella de carbono de una carga de cómputo se contabiliza como el producto de la energía consumida por la intensidad de carbono de la electricidad que la alimenta [1, 2]:

$$E_{\text{CO}_2} = W \cdot \text{PUE} \cdot I_{\text{grid}}, \quad (2)$$

donde W es la energía de cómputo (kWh), PUE (*Power Usage Effectiveness*) captura el sobre costo del centro de datos —refrigeración, distribución— (típicamente 1.4–1.6) e I_{grid} es la intensidad de carbono de la red ($\text{kgCO}_2\text{eq/kWh}$). Como PUE e I_{grid} son constantes para una infraestructura dada, la reducción *relativa* de emisiones coincide exactamente con la reducción relativa de energía:

$$\frac{\Delta E_{\text{CO}_2}}{E_{\text{CO}_2}} = \frac{\Delta W}{W}. \quad (3)$$

La ecuación (3) es la piedra angular de este trabajo: **ahorrar Joules es, uno a uno, ahorrar carbono**. Además, al evitar escalar al centro de datos, el tráfico resuelto en el borde no incurre en el factor PUE ni en la energía de transmisión de red, magnitudes que (2) amplifica sobre el gasto bruto de cómputo, y puede aprovechar generación renovable local [2].

5 Mecanismo de Decisión Energéticamente Óptima

Para asignar el cómputo intensivo de forma óptima y robusta, modelamos cada petición *sospechosa* como una etapa de un juego no cooperativo entre un Defensor y un Atacante [5]. El Defensor elige entre D_V (*Verde*: permanecer en el Edge, energía mínima) y D_A (*Activa*: escalar a la inspección Serverless). Su utilidad ante el par de acciones (a, d) integra seguridad, energía y daño:

$$U_D(a, d) = \alpha R(a, d) - \beta C_{\text{energy}}(d) - \gamma C_{\text{damage}}(a, d), \quad (4)$$

donde el término βC_{energy} penaliza *gastar* energía y el término γC_{damage} penaliza *no* gastarla cuando habría sido necesario. El equilibrio resuelve esta tensión.

El punto óptimo energético. Como el juego carece de equilibrio en estrategias puras, la solución es un equilibrio mixto obtenido por el *principio de indiferencia*.

Tabla 1: Top 10 de ataques a la nube y su mitigación con servicios AWS. SGE actúa como capa de eficiencia sobre los controles de inspección (filas destacadas).

#	Ataque	Descripción	Mitigación con servicios AWS
1	Credenciales/IAM comprometidas	Robo o abuso de claves y roles con exceso de privilegios.	IAM (mínimo privilegio), IAM Access Analyzer, IAM Identity Center + MFA, GuardDuty.
2	Configuraciones erróneas (S3 y otros)	Recursos o <i>buckets</i> expuestos públicamente por error.	S3 Block Public Access, AWS Config, Security Hub, Trusted Advisor.
3	Denegación de servicio (DDoS)	Saturación volumétrica o de capa 7 del servicio.	AWS Shield (Std/Adv), CloudFront, Route 53, reglas <i>rate-based</i> de WAF.
4	Inyección (SQLi/CMDi) y <i>prompt injection</i>	Payloads maliciosos que manipulan la consulta o el modelo de lenguaje.	AWS WAF (reglas gestionadas SQLi), API Gateway (<i>throttling</i>), Bedrock Guardrails, validación en Lambda. [SGE: filtrado estocástico]
5	Exposición/fuga de datos sensibles	Extracción de datos confidenciales no cifrados.	Amazon Macie, AWS KMS (cifrado), cifrado de S3, VPC endpoints.
6	Secuestro de cuenta y <i>phishing</i>	Robo de sesión o suplantación de identidad.	Amazon Cognito, IAM Identity Center + MFA, GuardDuty, CloudTrail.
7	APIs inseguras / autenticación rota	Autorización deficiente en endpoints expuestos.	API Gateway (<i>authorizers</i>), Amazon Cognito, Verified Permissions, AWS WAF.
8	Amenazas internas / abuso de privilegios	Uso indebido de accesos legítimos.	CloudTrail, IAM Access Analyzer, GuardDuty, AWS Config, mínimo privilegio.
9	Malware y <i>cryptojacking</i>	Compromiso de cómputo para minería o persistencia.	GuardDuty (Malware Protection), Amazon Inspector, Systems Manager Patch Manager.
10	SSRF y robo de metadatos	Abuso del servicio de metadatos de instancia.	IMDSv2, AWS WAF, <i>security groups</i> , segmentación de VPC.

La condición de indiferencia del Defensor se reduce a la igualdad

$$\underbrace{\beta(C_{\text{serverless}} - C_{\text{edge}})}_{\text{sobrecosto energético}} = \underbrace{\gamma(1 - q)C_{\text{damage}} + \Delta R}_{\text{daño evitado + detección}}, \quad (5)$$

cuya lectura sostenible es directa: en el equilibrio, el **Joule marginal invertido** en inspeccionar iguala exactamente a la **seguridad marginal obtenida**. El sistema no “sobre-inspecciona” —lo que malgastaría energía y carbono sin retorno de seguridad— ni “sub-inspecciona”. La probabilidad de escalado crece de forma monótona con la sospecha $s \in [0, 1]$ inferida por el Edge (Figura 1), concentrando el gasto en el tráfico de alto riesgo.

6 Arquitectura Verde de Dos Capas

Capa Edge (baja potencia). Función de paso rápido con costo $C_{\text{edge}} = 1$, cercana al usuario. Extrae métricas baratas y aplica *rate-limiting* que neutraliza los bots ruidosos sin invocar cómputo intensivo. Ejecuta además el motor de decisión de la Sección 5.

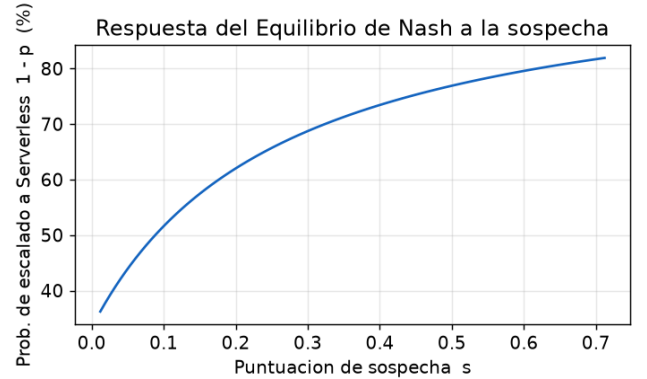


Figura 1: Política energética resultante: probabilidad de activar la costosa capa Serverless ($1 - p$) frente a la sospecha s . El gasto es despreciable para tráfico benigno y solo crece ante anomalías reales.

Capa Serverless (bajo demanda). Función efímera de inspección profunda que *solo* consume recursos al invocarse, con costo $C_{\text{exec}} = 15$ ($C_{\text{cold}} = 25$ tras inactividad [4]). Su naturaleza bajo demanda es en sí misma una elección de diseño sostenible: a diferencia de un servicio siempre activo, no consume energía en reposo. SGE explota esta propiedad activándola lo menos posible.

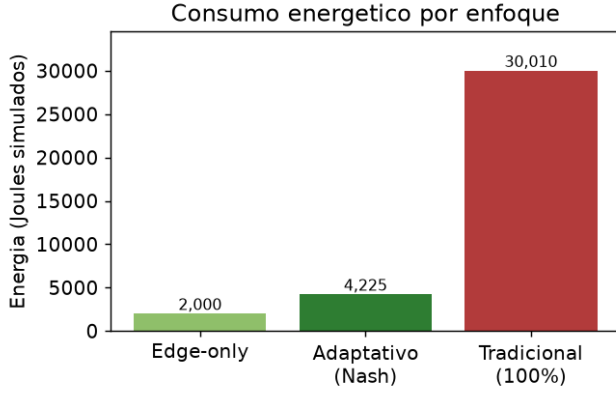


Figura 2: Consumo energético por enfoque. El Adaptativo se sitúa cerca del piso del Edge-only y muy por debajo de la inspección estática.

7 Metodología y Resultados

7.1 Diseño experimental

Simulamos $n = 2,000$ peticiones (semilla 7) con una mezcla realista: 80% legítimo, 15% bots ruidosos y 5% ataques dirigidos. Comparamos SGE (**Adaptativo**) contra dos líneas base: **Tradicional** (inspección del 100%) y **Edge-only** (nunca escala). Las métricas centrales son la energía consumida y la huella de carbono derivada; la seguridad (tasa de mitigación) actúa como restricción de calidad de servicio. La implementación se verificó con Ruff y 13 pruebas en Pytest.

7.2 Ahorro energético

De las 2,000 peticiones, solo 107 (5.35%) escalaron a la capa Serverless; el 94.65% restante se resolvió en el Edge al costo mínimo. En consecuencia, el consumo del Adaptativo se descompone en 2,000 Joules del Edge más 2,225 de las invocaciones profundas, totalizando **4,225** Joules frente a los **30,010** de la inspección estática: un **ahorro del 85.9%** (Figura 2). El Edge-only minimiza la energía pero deja pasar el 100% de los ataques avanzados, confirmando que la capa profunda es imprescindible y debe usarse —con criterio— para la fracción sigilosa.

7.3 Reducción de la huella de carbono

Aplicando el modelo de carbono (2), traducimos el ahorro energético a emisiones evitadas. Anclando de forma conservadora el costo de una inspección profunda tibia en $e_{\text{exec}} = 1 \text{ J}$ ($= C_{\text{exec}} = 15$ unidades del modelo), con $\text{PUE} = 1.5$ e $I_{\text{grid}} = 0.475 \text{ kgCO}_2/\text{kWh}$ (media mundial de la red [1]), la Tabla 2 y la Figura 3 muestran la huella anual para una API de 10^9 peticiones mensuales. SGE reduce las emisiones de 2.38 a 0.33 toneladas de CO_2 al año, **evitando $\approx 2.04 \text{ t CO}_2/\text{año}$ (85.9%)** —comparable a

Tabla 2: Huella de carbono anual por enfoque (API de 10^9 pet./mes; $\text{PUE} = 1.5$, $I_{\text{grid}} = 0.475$).

Enfoque	Mit. global	t $\text{CO}_2/\text{año}$
Tradicional (100%)	100%	2.38
Adaptativo (Nash)	94.38%	0.33
Edge-only (0%)	75.53%	0.16

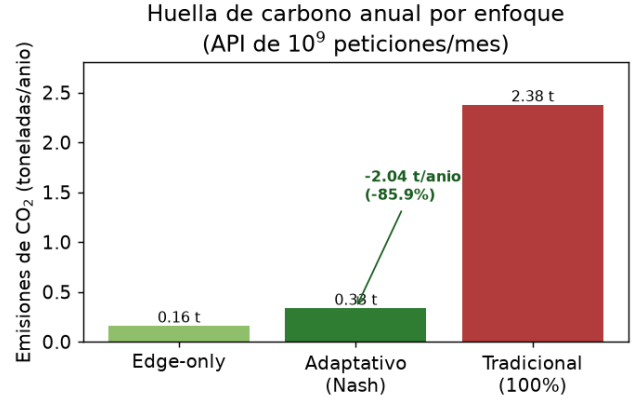


Figura 3: Huella de carbono anual por enfoque a escala de 10^9 peticiones/mes. El Adaptativo evita ≈ 2 toneladas de CO_2 al año frente a la inspección estática, conservando el 94.38% de mitigación.

retirar de circulación el equivalente a conducir un automóvil promedio unos 8,000 km— respecto a la inspección estática.

Dado que el ahorro es lineal en el volumen de tráfico, el beneficio ambiental escala con el tamaño del servicio (Figura 4): para APIs de hiperescala, las emisiones evitadas alcanzan decenas de toneladas anuales. Es importante subrayar que, si bien las magnitudes absolutas dependen de las hipótesis (e_{exec} , PUE , I_{grid}), la reducción *proporcional* del 85.9% —consecuencia directa de (3)— es independiente de ellas.

7.4 La frontera sostenibilidad–seguridad

La Figura 5 traza la frontera entre energía y seguridad al variar el valor asumido de la API. Permite al operador *elegir su punto de compromiso ambiental*: incluso en su configuración más segura, el Adaptativo consume un orden de magnitud menos de energía —y por tanto de carbono— que la inspección total.

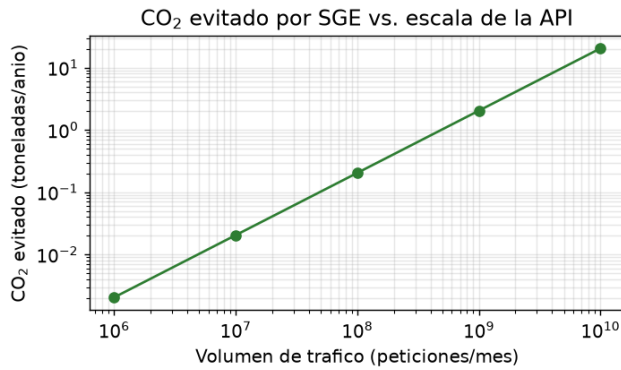


Figura 4: CO₂ evitado por SGE frente a la escala de la API (escala log-log). El beneficio ambiental crece linealmente con el volumen de tráfico.

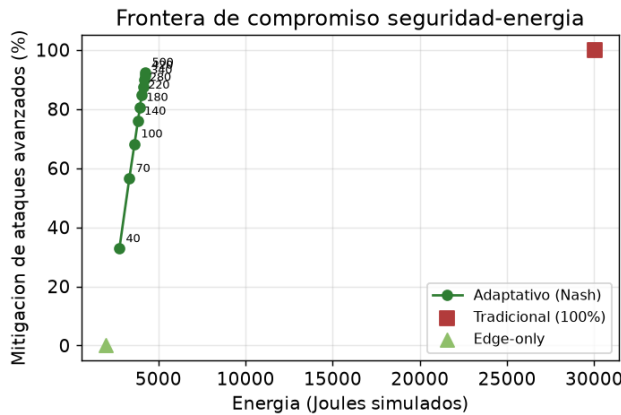


Figura 5: Frontera de compromiso sostenibilidad-seguridad. Cada punto es una política de equilibrio; el operador puede situarse donde prefiera sin abandonar el régimen de baja energía.

8 Análisis Técnico y Viabilidad como *Startup*

8.1 Manejo de tráfico masivo y carga de servidores

El valor operativo de SGE reside en su capacidad de *descarga* (*offload*). La ruta rápida del Edge tiene complejidad efectiva $O(1)$ por petición —extracción de métricas ligeras y, a lo sumo, la resolución de un juego bimatrix 2×2 cuya solución es cerrada (Sección 5) y se computa en microsegundos— de modo que el filtro puede saturar el ancho de banda de red sin convertirse en cuello de botella. Al resolver el 94.65% del tráfico en esta capa, el sistema actúa como un mecanismo de *load shedding* inteligente: protege a las funciones de inspección profunda del núcleo de los picos de tráfico y de los ataques de agotamiento de recursos, estabilizando la latencia de cola (*tail latency*) y permitiendo un escalado horizontal más económico. En

una arquitectura cloud típica, la capa Edge se materializa como *middleware* sin estado replicable (p. ej. un filtro en la pasarela de API o una función de borde), mientras que el motor de inspección —y su costo— se invoca bajo demanda.

8.2 Elección de lenguajes de programación

La naturaleza de dos capas sugiere una arquitectura *poliglóta* alineada con las prácticas cloud-native. Para la **pasarela/Edge** recomendamos **Go**: su modelo de concurrencia basado en *goroutines*, su recolector de basura de baja latencia, su compilación a un binario estático y su reducido consumo de memoria lo hacen idóneo para servicios de red de alto rendimiento —es el lenguaje de sistemas cloud de referencia (Kubernetes, Envoy, Traefik, Cloudflare). Para el **motor matemático** recomendamos **Python**: su ecosistema científico (NumPy, SciPy) permite expresar y calibrar el modelo de Nash con rapidez, y es el estilo del prototipo actual. La comunicación entre capas puede resolverse vía **gRPC** o, dado que el equilibrio 2×2 es de forma cerrada, reimplementando la ruta caliente directamente en Go y reservando Python para la calibración *offline* de parámetros y la experimentación. Esta separación —Go para el plano de datos, Python para el plano de análisis— maximiza tanto el rendimiento como la velocidad de iteración.

8.3 Viabilidad como modelo de negocio (B2B SaaS)

SGE es adaptable a un producto *B2B SaaS* con una propuesta de valor doble y poco habitual en el mercado de seguridad. **(1) Reducción de costos de infraestructura:** al recortar hasta un $\sim 86\%$ el cómputo de inspección, el cliente reduce directamente su factura de CPU/serverless, un argumento comercial inmediato y medible. **(2) Cumplimiento ambiental (ESG):** la reducción de la huella de carbono es *cuantificable y auditable* (Sección 7), lo que la convierte en una palanca concreta para los reportes de sostenibilidad —en alcance 2 y 3— exigidos por marcos regulatorios crecientes y por metodologías como la Software Carbon Intensity [11]. La entrega puede realizarse como *plugin* de pasarela o función de borde (p. ej. compatible con entornos serverless o de *edge*), minimizando la fricción de integración, con un modelo de tarificación por millón de peticiones o por ahorro compartido. El público objetivo natural son las empresas “API-first” y, de forma destacada, los proveedores de aplicaciones basadas en modelos de lenguaje, para quienes la inyección de prompts es una amenaza emergente y el costo por inspección es especialmente alto. **Conclusión de viabilidad:** SGE combina un ahorro de costos tangible con una ventaja de cumplimiento ESG y una defensa técnica diferenciada (la

política estocástica óptima), lo que lo hace viable como *startup*; los principales riesgos a validar son la precisión de detección en tráfico real y la facilidad de integración en las arquitecturas de los clientes.

9 Discusión

Los resultados muestran que la teoría de juegos es una herramienta eficaz de *ecodiseño*: permite capturar la mayor parte del valor de seguridad de la inspección exhaustiva con una fracción de su energía y, por (3), de su huella de carbono. El enfoque es sinérgico con otras estrategias verdes: al concentrar el 94.65% de las peticiones en el borde de baja potencia, habilita el aprovechamiento de energía renovable local y reduce la dependencia de centros de datos con alto PUE [2, 1].

Limitaciones. (i) Los costos energéticos son unidades relativas simuladas; una calibración con mediciones físicas de potencia refinaría las estimaciones absolutas de carbono [3]. (ii) PUE e I_{grid} varían por región y hora; una política *consciente del carbono* (*carbon-aware*) podría escalar preferentemente cuando la red es más limpia. (iii) La detección Serverless se idealiza como perfecta. (iv) Cada petición es una etapa independiente; un juego multi-etapa con estado permitiría políticas con memoria [5].

10 Conclusiones

Presentamos SGE desde la perspectiva de la sostenibilidad: una defensa perimetral que trata la inspección profunda como un recurso energético escaso y lo asigna mediante el equilibrio de Nash, igualando el Joule invertido con la seguridad obtenida. El resultado es un ahorro energético del 85.9% y una reducción proporcional de la huella de carbono —del orden de 2 toneladas de CO₂ anuales a escala de 10⁹ peticiones/mes— conservando un 94.38% de mitigación de amenazas. SGE demuestra que la ciberseguridad y la descarbonización no son objetivos opuestos: con el mecanismo de asignación adecuado, reforzar una refuerza la otra. Como trabajo futuro proponemos políticas *carbon-aware* que incorporen la intensidad de carbono en tiempo real y la medición física del consumo por capa.

Referencias

- [1] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey. *Recalibrating global data center energy-use estimates*. Science, 367(6481):984–986, 2020.
- [2] L. D. Radu. *Green Cloud Computing: An Energy-Efficient Ecodesign for IT*. Energies, 2017.
- [3] S. Murugesan and G. R. Gangadharan (eds.). *Harnessing Green IT: Principles and Practices*. Wiley, 2012.
- [4] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski. *The rise of serverless computing*. Communications of the ACM, 62(12):44–54, 2019.
- [5] T. Alpcan and T. Başar. *Network Security: A Decision and Game-Theoretic Approach*. Cambridge University Press, 2010.
- [6] M. H. Manshaei, Q. Zhu, T. Alpcan, T. Başar, and J.-P. Hubaux. *Game theory meets network security and privacy*. ACM Computing Surveys (CSUR), 45(3):1–39, 2013.
- [7] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. 2023–2025.
- [8] OWASP Foundation. *OWASP API Security Top 10*. 2023.
- [9] Cloud Security Alliance. *Top Threats to Cloud Computing: The Pandemic Eleven*. 2022.
- [10] Amazon Web Services. *AWS Well-Architected Framework: Security Pillar*. 2023.
- [11] Green Software Foundation. *Software Carbon Intensity (SCI) Specification*. 2024.